

Data preparation

Before training our model, we performed some data preparation steps, as follows.

Cleaning: Removing any invalid or missing data.

Preprocessing: Scaling the numerical features to ensure they have the same range, and normalising the data to improve the model's performance.

Feature engineering: Creating new features based on the existing data to improve the model's accuracy.

Model training and deployment

Once we had prepared our data, we used Amazon SageMaker's built-in XGBoost algorithm to train our binary classification model. We then deployed the trained model using SageMaker's model deployment service, which automatically scales the model to handle high volumes of traffic.

Integration with IoT devices

We then integrated the deployed machine learning model with IoT devices such as cameras and sensors to monitor passengers' movements and predict their arrival times at the gate.

Alerting system: To build an alerting system, we used Amazon Simple Notification Service (SNS) to notify gate agents and airline staff when passengers are predicted to be late.

Tracking system: To build a tracking system, we used facial recognition and other tracking technologies to locate passengers who are predicted to be late and escort them to the gate.

Overall, this implementation of predictive analysis using SageMaker can help airlines optimise their boarding process by predicting which passengers may be at risk of missing their flight and taking proactive measures to ensure they reach the gate on time.

Sample SageMaker code

SageMaker's built-in XGBoost algorithm was used to predict if a checked-in passenger will reach the boarding gate on time. Data was split

```
import boto3
import sagemaker

# Set up the SageMaker session and role next, Define the S3 bucket and prefix for storing the data and model
sagemaker_session = sagemaker.Session()
role = sagemaker.get_execution_role()
#
bucket = 'your_s3_bucket_name'
prefix = 'sagemaker/xgboost_model'

# Load the input data from S3
data_location = 's3://{}/{}/{}'.format(bucket, prefix)
data = sagemaker.inputs.TrainingInput(data_location, content_type='text/csv')

# Set up the hyperparameters for the XGBoost algorithm
hyperparameters = {
    'max_depth': '5',
    'eta': '0.2',
    'gamma': '4',
    'min_child_weight': '6',
    'subsample': '0.7',
    'objective': 'reg:squarederror',
    'num_round': '100'}

# Configure the estimator with the XGBoost algorithm and the hyperparameters
estimator = sagemaker.estimator.Estimator(
    image_uri=sagemaker.amazon.amazon_estimator.get_image_uri(
        sagemaker_session.boto_region_name, 'xgboost', '0.90-1'),
    role=role,
    instance_count=1,
    instance_type='ml.m5.large',
    output_path='s3://{}/{}/output'.format(bucket, prefix),
    sagemaker_session=sagemaker_session,
    hyperparameters=hyperparameters)

# Train the model with the input data
estimator.fit({'train': data})

# Deploy the trained model to an endpoint
predictor = estimator.deploy(initial_instance_count=1, instance_type='ml.t2.medium')

# Use the deployed model to make predictions and Delete the endpoint
test_data = '0,1,2,3,4,5,6,7,8,9' # Replace with your test data
prediction = predictor.predict(test_data)
|
predictor.delete_endpoint()
```

Figure 2: Sample SageMaker code

	Passenger Name	Time of Arrival	Passenger State	Shopping Duration	Distance from Gate	Gate Arrival Time	Flight Departure Time	Flight Delay	Walking Pattern	Hand Luggage	Terminal Congestion
1	Jayashree	07:20	Late	15	250	07:30	08:00	00:30	Slow	2	Moderate
2	Precila	08:05	On Time	10	150	08:15	09:00	00:00	Moderate	1	Low
3	Abdul	07:40	Late	20	300	07:50	08:30	00:15	Fast	3	High

Figure 3: Classification report

into training and validation sets, pre-processed with Pandas, and saved to S3. The model was trained and deployed to make predictions on new data. The model's accuracy was evaluated with a validation set, and metrics like confusion matrix and classification report were printed as shown in Figure 3.

As we saw in this article, by combining AWS IoT, AI, ML, and other services, airports can provide a smoother and more enjoyable passenger onboarding experience. This not only benefits passengers but also enhances airport operations, making them more efficient and cost-effective. **END** 🐧

Acknowledgements

I would like to thank Balakrishnan Srinivasan (CTO - hybrid cloud application modernization services, IBM India) for encouraging me to write this; Gopalakrishna U. (associate partner, IBM India) for providing me an opportunity to gain real-time experience on this subject; and Harsh Mehta (delivery manager, IBM India) who took the effort to help me release this article for publishing.

By: Venkateswaran N.

The author is a lead architect, containerization and modernization, AWS practice, IBM India.